

Robustness of a Random-Forest Classifier via OBDD

Project report, BDSS 2025/26, University of Verona

Francesco Simbola

July 2026

Abstract

A random forest classifies by letting many decision trees vote. It works well but it is a black box and testing it on samples says nothing about the inputs never tried. In this project I compile the forest into an OBDD, a compact diagram that describes the forest's certified answer for every possible input at once. On that diagram I then count inputs, rank features and measure how far a sample sits from changing class. I ran the full pipeline on all 54 provided models. Eighteen compile completely. For 34 I located the first tree prefix that fails. Only two never compile. Two findings stand out. What limits compilation is the number of encoded input bits and not the number of trees. The cheapest split heuristic beats the three sophisticated ones. Robustness runs on 50 of the 54 models. On 29 of them the median nearest-rival radius is a single bit flip. The implementation passes 110 tests, including a full brute-force comparison with zero mismatches.

Contents

1	Introduction	2
2	Notation at a glance	2
3	From the forest to a Boolean formula	2
4	Four questions answered on the diagram	5
5	Keeping the diagram small	6
6	How far a sample is from changing class	7
7	The code	7
8	The study on 54 models	9
9	One sample from start to finish	12
10	How I know it is correct	12
11	Limits and conclusion	13
A	Results for all 54 models	13
B	Robustness results for all 54 models	16

1 Introduction

A random forest is a committee of decision trees. Each tree asks a chain of yes-or-no questions of the form “is this feature below this threshold” and casts a vote. The class with the most votes wins. Testing the committee only shows how it behaves on the samples I try and nothing about the rest of the input space.

This project replaces spot checks with guarantees. I translate the whole forest into one Boolean formula and store it as an OBDD, an ordered binary decision diagram. An OBDD is a graph that reads the input bits one at a time in a fixed order and merges every repeated sub-question, which keeps it small. Once the forest lives in this form, three questions that are impossible to answer by sampling become single walks over the diagram. How many inputs does each class win beyond any doubt. Which input bits matter most for the outcome. Given one concrete sample, how many bits must change before the forest certifies a different class.

I follow the definitions and the required pipeline of the 2025/26 project assignment [1]. Section 2 collects the notation. Section 3 explains the translation. Sections 4 to 6 explain the three questions. Section 7 describes the code and Section 8 the experiments on all 54 models. Section 9 follows one real flower through the whole pipeline and Section 10 explains how I know the result is correct.

2 Notation at a glance

The same small set of symbols appears throughout the report. Table 1 keeps their meaning in one place. I use $\mathcal{F}$ for the forest and F for the number of input features.

Symbol	Meaning
$\mathcal{F} = \{T_0, \dots, T_{D-1}\}$	forest made of D decision trees
F	number of input features
$\mathcal{L}$	set of class labels
$\text{cells}(f)$	number of cells induced on feature f
b_f	number of bits used for feature f
B_j^f	bit j of the cell index of feature f
X_f	integer encoded by the bits of feature f
τ	cell index of a tree threshold
LC^ℓ	running vote counter of class ℓ
R_t	trees still to vote after tree T_t
A_t	accumulator after tree T_t
B	maximum desired OBDD size before a split
ρ	partial assignment that identifies one slice
Win_ℓ	inputs certified as class ℓ
$\hat{x}$	binary encoding of one sample
$\text{rob}_\ell(\hat{x})$	distance from $\hat{x}$ to class ℓ

Table 1: Notation used in the report.

3 From the forest to a Boolean formula

The translation happens in four steps. I first replace real numbers with a finite set of intervals, then write each interval as bits, then teach the formula to count votes and finally chain the trees together.

Step 1, from real numbers to cells. A tree never looks at the exact value of a feature. It only checks whether the value is below certain thresholds, so two values that fall between the

same pair of thresholds are treated identically by every tree in the forest. This means I can replace the real line with a finite list of intervals, the way age brackets replace exact ages on a form. I collect every threshold that any tree uses on a feature, sort them and call *cells* the intervals between consecutive thresholds (Def. 1). A feature with k thresholds has $k + 1$ cells. The cells are closed on the right because the tree tests use $\leq$. Prop. 1 states that classification only depends on the cell and I checked this on real data. For every training and test sample of four datasets, replacing each value with any other value from the same cell leaves the prediction of `sklearn` unchanged.

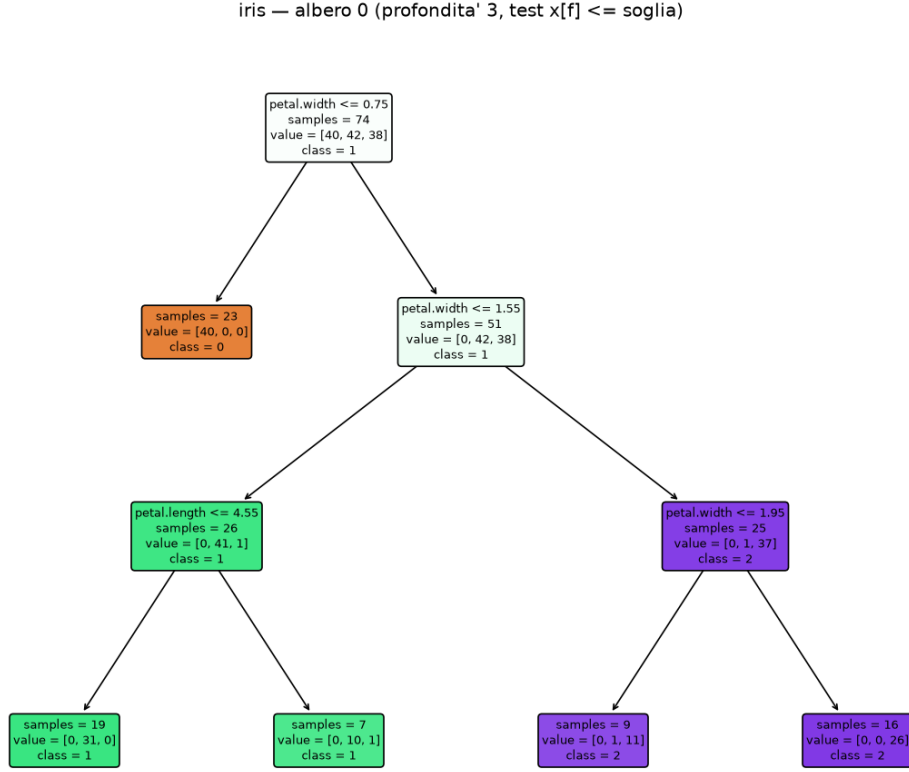


Figure 1: The first tree of the iris forest. Every internal node asks whether a feature is below a threshold and these thresholds are exactly the cell borders that step 1 collects.

Step 2, from cells to bits. Each cell gets a number starting from zero and the number is written in binary. A feature with up to 2^b cells needs b bits, so a feature with 32 cells costs 5 Boolean variables (Def. 3). The assignment explicitly forbids the naive alternative of one variable per cell, which would cost 32 variables plus a constraint saying exactly one of them is true. When the number of cells is not a power of two, some bit patterns are left over and I read them as the last cell (Remark 1).

For feature f the number of bits and the encoded integer are

$$b_f = \lceil \log_2 \text{cells}(f) \rceil \quad X_f = \sum_{j=0}^{b_f-1} 2^j B_j^f.$$

This is the complete binary encoding. It needs no auxiliary constraint because every bit pattern already denotes one integer.

Every tree question now becomes a question on bits. The test “is the value below threshold number τ ” turns into “is the cell number at most τ ”. Comparing two binary numbers is easy if the walk starts from the most significant bit, because the answer is settled at the first position where they differ. This gives a small formula called $nt(f, \tau)$ whose size grows only linearly with the number of bits (Def. 4, Prop. 2). Figure 2 shows one drawn as a diagram. My tests try every possible bit pattern against plain integer comparison and the two always agree.

Writing τ_j for bit j of the fixed threshold, I build the comparator with

$$\psi_{-1} = \top \quad \psi_j = \begin{cases} \neg B_j^f \wedge \psi_{j-1} & \text{if } \tau_j = 0 \\ \neg B_j^f \vee \psi_{j-1} & \text{if } \tau_j = 1 \end{cases} \quad nt(f, \tau) = \psi_{b_f-1}.$$

Each step adds only one Boolean operation. The final formula therefore has size linear in b_f .

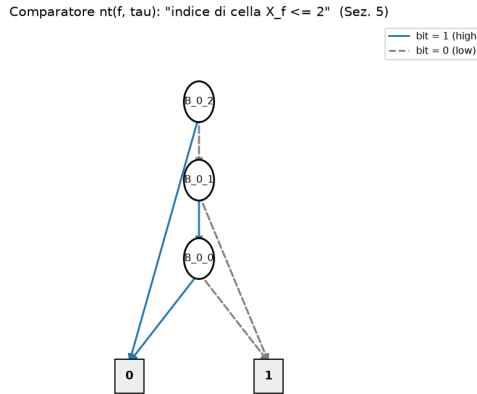


Figure 2: The comparator for a feature with six cells, asking whether the cell number is at most 2. Each circle reads one bit. Solid arrows mean the bit is one, dashed arrows mean zero and the squares are the two possible answers.

Step 3, counting votes like an election. The formula must now count votes and the key idea comes from elections. A candidate who has more votes than all the others could ever reach has already won, no matter how the remaining voters decide. With D trees the safe threshold is $W = \lfloor D/2 \rfloor + 1$ votes (Def. 5).

Every class gets a small binary counter. After tree number t has voted, $R_t = D - 1 - t$ trees still have to vote. Class s has *clinched* the election when

$$LC^s \geq LC^\ell + R_t \quad \text{for every rival class } \ell,$$

where LC^s is the vote count of s and LC^ℓ the count of the rival (Def. 9). In words, even if every remaining tree voted for the same rival, that rival could not catch up. The counters have one bit more than strictly needed so that the sum $LC^\ell + R_t$ never overflows. In the first half of the election nobody can have clinched yet and the code returns false immediately in that case without building anything.

Each tree then contributes a rule with two branches (Def. 10). If somebody has already won, the win flags are frozen and the counters are released. Released variables literally disappear from the diagram because their value no longer matters. If nobody has won yet, the tree votes. The path through the tree leads to a class. Either that class clinches on the spot and its flag is set, or its counter grows by one while all the others are copied unchanged.

One subtlety deserves a paragraph because it is the reason the whole approach makes sense. For two classes the certificate is exact, since the majority class always clinches. For three or more

classes it is not and this is intentional (Prop. 3). The assignment gives the example I also keep as a unit test. Seven trees vote A,A,A,B,C,B,C, so A wins the count 3 to 2 to 2, yet after A's third vote four trees remain and either rival could in principle still reach 4. No class ever leads beyond doubt, so no flag is set. The diagram certifies the winner that is *guaranteed whatever the remaining trees do*. A guarantee is exactly what a robustness question needs. An input without a guaranteed winner is simply not certified.

Step 4, chaining the trees. The tree rules are applied one after another. I keep a running formula, called the accumulator, that describes every possible state of the counters after the trees processed so far. Adding a tree means combining the accumulator with the tree's rule and then forgetting the old counter values, which is a standard OBDD operation (Def. 14). Forgetting loses nothing because each rule determines the new state completely from the old one (Prop. 4). At the end I ask the accumulator one last question per class, namely for which inputs is your flag set. This gives one diagram Win_ℓ per class over the input bits alone. Its satisfying assignments are exactly the inputs certified as class ℓ and two classes never share an input. Figure 3 shows such a diagram for a toy model small enough to read.

Let $\mathbf{x}$ contain the input bits, let $\mathbf{c}$ contain the current counters and flags and let $\mathbf{c}'$ contain their next copy. The complete update is

$$A_t(\mathbf{x}, \mathbf{c}) = [\exists \mathbf{c}' (A_{t-1}(\mathbf{x}, \mathbf{c}) \wedge \Phi_t(\mathbf{x}, \mathbf{c}, \mathbf{c}'))] \{\mathbf{c}' \mapsto \mathbf{c}\}.$$

The quantifier deletes the state that has just been consumed. Renaming makes the next state ready for the following tree. At the end I extract one class with

$$\text{Win}_\ell(\mathbf{x}) = \exists \mathbf{c} (A_{D-1}(\mathbf{x}, \mathbf{c}) \wedge \text{label_win}_\ell).$$

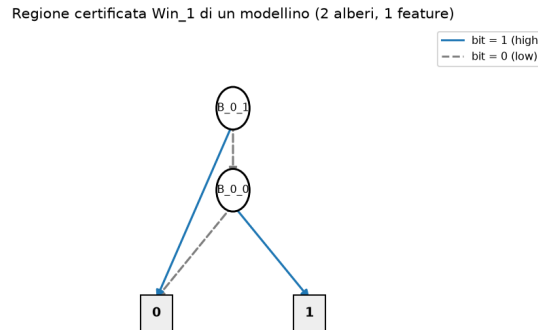


Figure 3: The certified region of one class for a toy model with two trees and one feature. Real models produce the same kind of diagram with thousands of nodes.

4 Four questions answered on the diagram

All four readouts below are the same algorithm dressed differently. Walk the diagram from the bottom to the top, combine the two children of every node with some operation and remember results so that shared nodes are computed once. The code has exactly one such walker, the function `bottom_up_pass`. Every readout just plugs in different operations.

Counting. Adding the two children and doubling across skipped bits counts the satisfying inputs (Theorem 1). This answers “how many inputs does class ℓ win beyond doubt”. I delegate the plain count to the library and my own walker reproduces it in the tests as a cross check.

For a node u at level i with children at levels j_0 and j_1 , the exact recurrence is

$$C(u) = 2^{j_0-i-1}C(\text{low}(u)) + 2^{j_1-i-1}C(\text{high}(u)), \quad C(\perp) = 0, \quad C(\top) = 1.$$

The powers of two count the variables skipped by an edge. Changing the values stored at the terminals and the two combining operations turns this same pass into volume, Shapley values and the counts needed by the p-value.

Volume. If instead of adding ones I add weights, one weight per bit value, the count turns into a weighted volume. With all weights equal to one half the result is the certified fraction of the bit patterns, the uniform measure over patterns (Prop. 5). This is a declared convention and not a geometric volume. Cells have different real widths and the two border cells of each feature stretch to infinity, yet every pattern counts the same. The leftover patterns of a feature whose cell count is not a power of two all read as its last cell, so that cell weighs a little more, the bias Remark 8 anticipates.

Feature importance by Shapley values. The Shapley value of a bit measures how much that bit contributes, on average over all coalitions, to fixing the outcome. Prop. 7 gives a closed formula that only needs, for every satisfying input, how many of its bits are zero. I get those numbers by running the walker with small polynomials instead of numbers, one pass per bit (Prop. 8). The tests verify the closed formula directly and also check the classic sanity identity, namely that all values sum to a known total.

Feature importance by p-values. The second ranking asks, for each bit, whether setting it to one is unusually associated with being certified and scores this with the tail of a hypergeometric distribution (Def. 18). The numbers involved are astronomical, the population is all 2^n inputs, so everything runs on logarithms of factorials. When the exact sum would need too many terms I switch to a controlled normal approximation. Only the resulting ranking of bits is used downstream. The tests compare the tail against exact rational arithmetic on the sizes where the exact sum is feasible.

5 Keeping the diagram small

Some forests are too big to compile in one piece. The remedy of Def. 19 is a size budget B . I grow the accumulator tree by tree and as soon as it exceeds B nodes I choose one input bit and split the problem in two, one copy with that bit fixed to zero and one with it fixed to one. Each copy continues on its own and may split again. Think of cutting the input space into slices and compiling each slice separately. The final slices partition the space, so any count is recovered by summing over slices. Each slice is exactly the restriction of the full diagram with no approximation involved (Prop. 10).

Two details matter in practice. First, after fixing a bit I must also fix it inside all the tree rules that are still to come, otherwise the bit would silently reappear as a free variable and every later count would be wrong. My tests check the summing property for this exact reason. Second, when all input bits are already fixed but the diagram still exceeds a tiny budget, splitting is impossible. Prop. 10.4 guarantees the diagram is about to collapse on its own, so I finish the slice ignoring the budget. Any other reading of the pseudocode loops forever.

Which bit to split on is a policy choice and the assignment prescribes five (Def. 21). Three are semantic. Information gain picks the bit that splits the certified inputs most evenly, the Shapley criterion picks the bit with the strongest contribution and the p-value criterion picks the bit with the strongest association. Two are structural and ignore meaning entirely. One picks the variable of a random node of the diagram, the other picks the variable that labels the most nodes, called `modal_var`. Section 8 shows the surprising ranking among the five.

6 How far a sample is from changing class

This is the most concrete question of the project. Take a real sample and a class ℓ . How many bits of the sample’s encoding must flip, at minimum, so that the forest certifies ℓ . Def. 20 calls this the robustness radius. It is zero when the sample is already certified ℓ and infinite when no input at all is certified ℓ .

Theorem 2 reduces the radius to a shortest path in the diagram. Run the usual bottom-up walker but replace sum with minimum and product with addition. At each node, taking the branch that agrees with the sample’s bit costs nothing and taking the other branch costs one flip. Bits the diagram never reads cost nothing either, since they can stay as they are. I run this directly on the slices from the previous section, which still contain counters and flags, so a few extra weights forbid cheating. Paths that set a rival’s flag cost infinity, paths that leave the target flag unset cost infinity and counter bits are free because they are not part of the input (Prop. 11). The answer for the whole space is the minimum over slices, after paying one flip for every fixed bit of a slice that disagrees with the sample.

For an input node u that reads bit x_i , the distance recurrence is

$$d(u) = \min([0 \neq \hat{x}_i] + d(\text{low}(u)), [1 \neq \hat{x}_i] + d(\text{high}(u))), \quad d(\perp) = +\infty, \quad d(\top) = 0.$$

An edge that disagrees with the sample costs one. An edge that agrees costs zero. This is why the value at the root is the minimum number of bit flips.

The study aggregates this per-label radius in two ways. The untargeted value of a sample is the distance to the nearest rival certified region, the smallest radius among the rival labels. The escape value is the distance to the complement of the sample’s own region, computed with a variant of the same pass that only forbids turning the own flag on. Escape never exceeds the rival distance. On two-class models with an odd number of trees the two coincide, because there every input is certified and losing the certificate means handing it to the rival.

Beyond the number, I also reconstruct a *witness*, meaning the actual list of bits to flip. After the walker has filled in all the minimal costs I walk back down from the root always choosing the cheaper branch and record every disagreement. The output states which features those bits belong to, so a radius of 2 comes with a statement like “move sepal width to this interval and petal length to that interval”.

7 The code

The package `src/obddrf` has one module per step of the pipeline, plus a subpackage for the experiments. Every function carries a one-line comment naming the definition of the assignment it implements, so code and specification can be compared line by line. Table 2 is the map.

Module	Assignment part	Contents
<code>bdd_backend</code>	constraint 3	backend choice, CUDD preferred, <code>autoref</code> fallback
<code>config</code>	Section 14	all tunable parameters with the rationale in comments
<code>forest_io</code>	infrastructure	model loading, CSV splits, label-encoding validation
<code>discretize</code>	Sections 3–4	cells, bit encoding, conversions in both directions
<code>comparator</code>	Section 5	the comparator $nt(f, \tau)$
<code>counters</code>	Sections 6–7	counters, clinch, the per-tree rule
<code>compose</code>	Section 8	variable order, chaining, the Win_ℓ diagrams
<code>selectors, frontier</code>	Section 12	the five split policies and the budget algorithm
<code>readouts</code>	Sections 9–11	the walker and its four uses
<code>robustness</code>	Section 13	radius, witness, compiled slices
<code>experiments/</code>	Section 14	campaign runner, batch scripts, analysis, figures

Table 2: Package layout.

How to read it. The modules read in the same order as this report. `discretize.py` builds the cell structure and converts samples in both directions, with a test proving the round trip is lossless. `comparator.py` is one function. `counters.py` follows the definitions top to bottom, from the winning threshold to the function `tree_relation` that assembles the per-tree rule as one line mirroring Def. 10 verbatim. `compose.py` contains the chaining loop and fixes the variable order once for everyone. `readouts.py` is the walker plus its four wrappers. `frontier.py` keeps a stack of slices and asks `selectors.py` for the split bit when the budget is exceeded. `robustness.py` offers the same computation twice. The plain route, `min_switch`, walks live diagram objects and is what the test oracles use. The fast route flattens each slice once into plain arrays, class `CompiledLeaf`, so that evaluating one sample is a simple loop over integers. On the `wdbc` model the fast route brought 50 samples from 749 down to 35 seconds. In `experiments/`, `run_grid` owns the campaign, one subprocess per configuration with a hard kill and a resume flag. `run_robustness_batch` measures the radius over the test sets, `run_readouts` produces counts and rankings and `analyze_grid` turns the campaign log into the tables of Section 8.

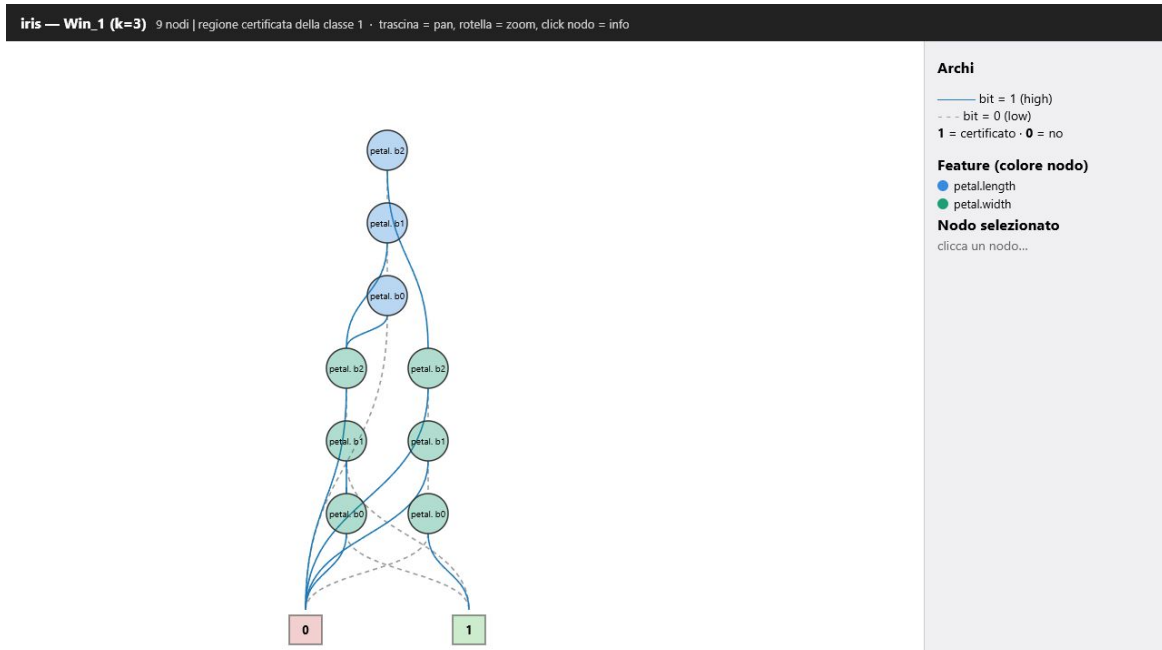


Figure 4: A real compiled OBDD from the iris model. The diagram represents the class-1 certified region of the first three trees and contains 9 internal nodes. A small real prefix is shown because complete real forests can produce diagrams with thousands of nodes. Solid edges read bit 1 and dashed edges read bit 0.

The backend. The assignment prescribes the `dd` library with the CUDD engine. On the Windows machine of the experiments the CUDD extension cannot be built, so the code runs on `dd.autoref`, the pure-Python engine with the same interface. The backend module tries CUDD first and falls back automatically, so moving to Linux would switch engines without touching anything else. Only speed is affected. The results are identical by construction and the test suite passes on either engine.

Three lessons the code learned the hard way. The `dd` library represents negation with so-called complemented edges, where a negated reference to a node also negates its children. An early version of the witness walk read the children directly and produced wrong radii, in one case a radius of 0 where the correct answer was infinite. The disagreement with the slow validated

route exposed the bug. The fix negates the children of negated references and a permanent regression test now compares the two routes on a real model.

The resource monitor checks time limits between trees, so a single oversized tree rule can stall for hours without ever reaching a check, which happened on the 26-class model `letter`. Every configuration therefore runs in its own subprocess and is killed from outside sixty seconds past its deadline. A run still alive at that point cannot finish anyway.

The 54 datasets do not encode their class labels uniformly. Two are inverted with respect to alphabetical order and one numbers classes by frequency. The loader tries five candidate encodings and keeps the one that reproduces the accuracy stored with the model. All 54 resolve.

Listing 1: The edge costs of the robustness walk as implemented in `min_switch`.

```
def weight(i, b):
    var = ordered[i]
    if var in input_set:          # flipping a cell bit costs one
        return 0.0 if b == sample_bits[var] else 1.0
    if var in flag_of:           # target must win and rivals must not
        if flag_of[var] == target_label:
            return 0.0 if b else inf
        return inf if b else 0.0
    return 0.0                   # counter bits are free
```

8 The study on 54 models

Setup. The repository ships 54 tuned forests with up to 100 trees of depth 3 to 10. Encoded, they range from 9 to 816 input bits with a median of 66 and from 2 to 26 classes. The instructor left the experimental parameters to me, so I fixed them after a calibration run. Budgets of 1 000, 10 000 and 100 000 nodes, because the small one forces splits on most models and beyond 10^5 nothing changes. A time limit of 300 seconds per configuration with the hard kill at 360, because a shorter limit cut off models that finish comfortably, for instance the full `banknote` forest needs 42.5 seconds. A cap of 20 million diagram nodes as a portable stand-in for memory. Growing tree prefixes of 5, 10, 20, 30, 50 and all trees. When a prefix fails, a bisection narrows the gap to an adjacent pair, a prefix that completes and the next one that fails. Robustness runs on 50 test samples per model. The full grid produced 3 668 configurations of which 2 329 completed. The failures were 1 065 timeouts, 270 hard kills and 4 process errors.

Experimental protocol. I treat one model, one tree prefix, one budget and one selector as one configuration. Each configuration starts in a fresh process with a new OBDD manager. I use seed zero for the random selector. A configuration is complete only when every frontier slice reaches its final tree inside the resource limits. A timeout, a hard kill or a process error is recorded as a failure and never contributes runtime or size statistics to a successful group.

For each model I test prefixes of 5, 10, 20, 30 and 50 trees together with the full forest. Values larger than the forest are replaced by its true size. I stop increasing the prefix after the first failure for each budget and selector. I then use bisection between the largest successful prefix and the smallest failed prefix. This gives adjacent values k and $k + 1$ for every partial model.

I compare budgets only on the same model, prefix and selector completed by all three budgets. I compare selectors only on the same model, prefix and budget completed by all five selectors. I also require at least one real split when I study the selectors because the choice rule has no effect when the frontier stays in one slice. For robustness I use the first 50 rows of the held-out test split in their stored order. I compute the radius toward every rival class and take the smallest rival radius as the untargeted value.

Where compilation stops. Eighteen models compile in full, among them forests with 68 and 75 trees. Thirty-four compile up to a prefix and for each of them I verified the adjacent pair, the prefix that completes and the next one that fails. `ann-thyroid` succeeds at 9 trees and fails at 10 out of 100, `bupa` at 27 versus 28, `diabetes` at 17 versus 18, `shuttle` at 11 versus 12. Only `letter` and `texture` fail from the very first tree. The striking part is what does *not* predict the wall. Tree count does not, since 75 trees can pass while `pima` with 16 trees stops at 9. What predicts it is the number of encoded input bits. Every model that compiles in full has at most 66 bits and every model above that walls. Figure 5 shows the pattern.

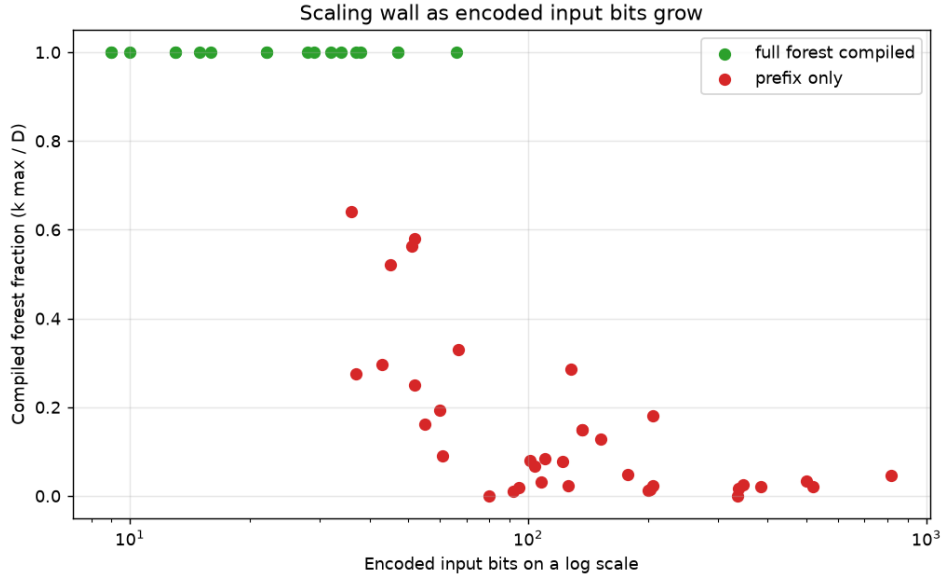


Figure 5: Fraction of the forest compiled against encoded input bits on a log scale. Green points compile in full and red points stop at a prefix.

What the budget buys. On the 250 configurations that all three budgets completed and where at least one split happened, the small budget behaves exactly as Remark 12 predicts. It keeps the diagram small with a median peak of 1 779 nodes against about 11 000. In exchange it fragments the space into a median of 142 slices against 1 and takes 4.30 seconds against under one. Table 3 has the numbers.

Budget B	Slices (median)	Peak nodes (median)	Runtime (median)
1 000	142	1 779	4.30 s
10 000	1	10 269	0.92 s
100 000	1	11 584	0.77 s

Table 3: The three budgets on the 250 split-active configurations completed by all.

Sensitivity to the budget. The matched comparison in Table 3 changes only B . Moving from 1 000 to 10 000 nodes reduces the median number of slices from 142 to 1. The median peak grows from 1 779 to 10 269 nodes and the runtime falls from 4.30 to 0.92 seconds. Moving again to 100 000 nodes leaves the median at one slice. The peak grows slightly and the runtime changes little. The main conclusion is therefore stable. A very small budget saves diagram space but repeats the work over many slices. A medium budget removes most of that repetition. The

larger budget is still useful on the hardest models because it raises the number of full forests completed.

Which split policy wins. The policy only acts when a split happens, so the fair comparison restricts to the 101 configurations that all five policies completed with at least one split. The cheapest policy wins outright. `modal_var`, which just picks the most frequent variable in the diagram, produces 9 slices in 3.33 seconds median. The three semantic policies all produce 21 slices in 7 to 9 seconds and the random baseline produces 21 in 11.30. The explanation is not mysterious. The semantic policies look at the certified inputs, but during compilation most of the diagram’s size lives in the counter variables that those policies cannot see. A bit that splits the inputs beautifully may not shrink the diagram at all, whereas the most frequent variable attacks size by definition. The assignment itself warns that no semantic criterion is guaranteed to help and my data confirm it.

Policy	Slices (median)	Runtime (median)	Completion rate
<code>modal_var</code>	9	3.33 s	65%
<code>shapley</code>	21	8.72 s	62%
<code>infogain</code>	21	7.16 s	63%
<code>pvalue</code>	21	7.16 s	63%
<code>random_node</code>	21	11.30 s	64%

Table 4: The five split policies on the 101 split-active configurations completed by all.

Counts and rankings. The readout batch covers 33 of 54 models, the ones whose diagram compiles in one piece. On `iris`, whose 16 input bits give 65 536 possible inputs, 56 144 are certified, which is 85.7% of the space, split into 7 118 for `Setosa`, 19 898 for `Versicolor` and 29 128 for `Virginica`. The p-value and Shapley rankings both put bits of petal width first, which is exactly the feature botanists would name. Information gain promotes a sepal width bit instead, because it rewards splitting the space evenly rather than deciding the outcome. The rankings answer different questions and their disagreement is itself informative.

Robustness over the test sets. The final robustness batch covers 50 of 54 models with up to 50 test samples each. This gives 2 304 samples in total. A harness bug found during the final review had first misreported 14 completed runs as timeouts. Their per-sample files were complete on disk. The child process was blocking while returning every row to the parent. The fix sends only a compact summary. I then repeated the campaign with a 600-second limit. Six of the eight runs that had exceeded 300 seconds completed between 348 and 445 seconds. Only `factors` and `karhunen` still reach the 600-second limit. Both have 10 classes and more than 500 encoded input bits. The models `letter` and `texture` have no compilable prefix.

Of the 2 304 samples, 2 229 are certified. The other 75 stay out of the distribution because their nearest-region distance measures the cost of gaining a certificate rather than losing one. Every certified sample counts exactly once. Rivals tied at the same distance do not double count. The reported median is the statistical one. Across the 50 models the median untargeted radius per model spans 1 to 6 bit flips. Twenty-nine models sit at median 1. The most robust are `seismic_bumps` with median 6 followed by `appendicitis` and `spectf` with 4. Figure 6 shows the distribution over the certified samples. The escape distance is strictly smaller than the nearest-rival distance for 329 samples. It also lowers the model median in 8 of the 50 models. This confirms that losing a certificate and reaching a certified rival are different questions in practice.

Appendix B reports the sample count, certified count and both model medians for every dataset.

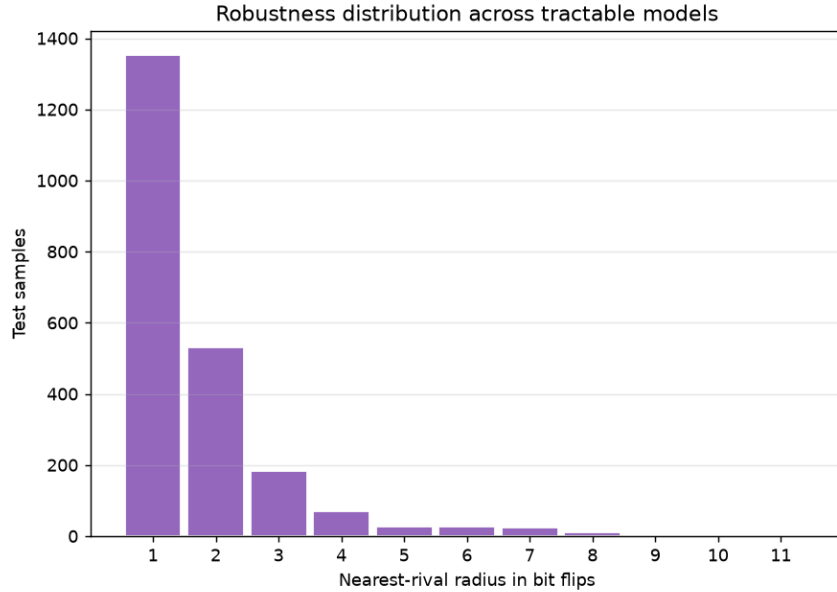


Figure 6: Distribution of the radius over all tractable models with up to 50 test samples each.

9 One sample from start to finish

Row 0 of the `iris` test split is a Setosa flower with sepal length 4.4, sepal width 3.0, petal length 1.3 and petal width 0.2. The forest has 26 trees and encodes to 16 input bits, namely 4, 3, 5 and 4 bits for the four features. The sample lands in cells (0, 5, 0, 0). For sepal width, five of the six collected thresholds lie below 3.0, which puts the value in cell 5, the interval (2.95, 3.15].

All 26 trees vote Setosa. The very first tree, the one in Figure 1, settles its vote at the root, where petal width at most 0.75 holds and leads straight to a pure leaf. The winning threshold is $W = 14$, the clinch fires as soon as Setosa’s lead is safe and every later tree just passes the verdict along. At budget 10^4 the whole forest compiles into a single slice with a peak of 1312 nodes and at budget 10^3 it splits into four.

The robustness output reads radius 0 toward Setosa with nothing to flip and radius 2 toward each rival. Toward Versicolor the witness flips one sepal width bit and one petal length bit. The first moves sepal width from cell 5 to cell 1, the interval (2.45, 2.55]. The second moves petal length from cell 0 to cell 8, the interval (4.45, 4.55]. Note that one flipped bit can jump several centimetres, because the radius counts flips in the encoding as Def. 20 prescribes and not distance in the original units. Both rivals sit at radius 2, so this flower is equally far from either boundary.

10 How I know it is correct

The suite contains 110 tests and each step is checked against an oracle that shares no code with the step itself. The discretization tests prove the conversion from reals to cells to bits and back is lossless on entire datasets and that the prediction of `sklearn` does not change when a value is replaced by a representative of its cell. The comparator tests enumerate every bit pattern. The counter tests replay hand-built vote sequences against a small independent simulator of the election, including the seven-vote counterexample of Prop. 3. The decisive oracle sits at

the chaining step. For hundreds of random and real inputs, the class certified by the final diagram must equal the output of the independent simulator and on two-class datasets also the plain majority vote. The readout tests compare every count, volume, Shapley value and p-value against brute-force enumeration. The slicing tests verify that slices sum back exactly to the whole for all five policies. The robustness tests compare radii against an exhaustive search over all inputs and check that flipping the witness really lands in the target region. The escape distance has its own exhaustive oracle and must also never exceed the nearest rival distance. A batch regression verifies that tied rivals count once and that summaries use the statistical median.

Two checks go further. On a four-tree iris forest I enumerated all 512 possible inputs and compared radius and witness of the fast route against total brute force for every input and every class. That is 1536 comparisons with zero mismatches. Across the real result files, every two-class model with an odd number of trees certifies 100% of its test samples. The theory demands exactly that and most conceivable bugs in the counters or the chaining would break it. All 15 such models comply.

11 Limits and conclusion

The limits are easy to state. The engine is pure Python and the C engine CUDD would push the walls outward the day the code runs on Linux with no changes needed. Certification with three or more classes is deliberately incomplete, as discussed. The volume is the uniform measure over bit patterns rather than a geometric one. The last cell receives extra weight when unused patterns are clamped to it. The forest of the assignment votes with hard leaf labels while the `predict` of `sklearn` averages the probability estimates of the trees. Measured over the 54 test sets the two rules disagree on 504 of the 37 950 samples, so I never claim identity with `sklearn`'s output. A separate and much smaller precision effect exists. `sklearn` casts inputs to 32-bit floats internally while the pipeline compares the stored 64-bit values, which moves a threshold-grazing value across a branch in 438 of roughly two million sample-tree pairs and flips the final hard vote on 5 samples. The readout batch compiles the monolithic diagram, so its coverage stops at 33 of 54 where aggregating over the slices of the frontier could reach further. Robustness covers 50 of 54 and its distributions rest on up to 50 samples per model.

What the project delivers is the complete chain the assignment asks for. A forest becomes a diagram and the diagram becomes counts, rankings and radii, with every step validated against an independent oracle. The study adds three facts. Feasibility is governed by encoded input bits, with the practical border near 66 bits on this engine. When memory forces splits, the cheapest structural policy beats the three semantic ones. And on more than half the models within reach, a typical test sample is a single bit flip away from losing its certified class, which makes the certificates concrete enough to say something about individual predictions.

References

- [1] Pietro Sala. *Project Assignment 2026. Robustness of a Random-Forest Classifier*. BDSS 2025/26, University of Verona, Version 0.1, 11 June 2026.

A Results for all 54 models

Table 5 reports the full-forest characteristics and the largest tree prefix completed by at least one tested configuration. The bit count is computed from all thresholds of the full forest. The value k is the largest completed prefix. Full means $k = D$. Partial means that k completes and $k + 1$ is the first failed prefix. None means that even the first tree does not complete.

Dataset	Classes	Features	Bits	Trees	Depth	Best k	Result
ann-thyroid	3	21	61	100	9	9	partial
appendicitis	2	7	38	68	6	68	full
banknote	2	4	32	44	10	44	full
biodegradation	2	41	206	88	10	2	partial
breast-cancer	2	9	22	64	3	64	full
breast-cancer-wisconsin	2	30	137	40	9	6	partial
bupa	2	6	37	98	8	27	partial
cloud	4	7	43	44	10	13	partial
diabetes	2	8	52	68	8	17	partial
ecoli	5	7	36	39	9	25	partial
factors	10	216	816	44	10	2	partial
fourier	10	76	501	90	8	3	partial
glass	5	9	60	67	9	13	partial
glass2	2	9	29	12	5	12	full
haberman	2	3	15	44	4	44	full
heart-c	2	13	37	33	4	33	full
heart-h	2	13	34	39	4	39	full
heart-sa	2	9	45	48	4	25	partial
ionosphere	2	34	152	70	6	9	partial
iris	3	4	16	26	5	26	full
karhunen	10	64	520	96	10	2	partial
letter	26	16	80	100	10	0	none
lupus	2	3	13	28	4	28	full
magic	2	10	104	44	10	3	partial
mammo	2	13	13	12	5	12	full
messidor	2	19	122	64	10	5	partial
mofn-3-7-10	2	10	10	75	10	75	full

Table 5: Characteristics and scaling result for the first 27 models.

Dataset	Classes	Features	Bits	Trees	Depth	Best k	Result
new-thyroid	3	5	28	51	8	51	full
pendigits	10	16	126	44	10	1	partial
phoneme	2	5	55	56	10	9	partial
pima	2	8	51	16	9	9	partial
ring	2	20	178	82	10	4	partial
segmentation	7	19	101	75	6	6	partial
seismic_bumps	2	18	67	88	7	29	partial
shuttle	7	9	52	19	10	11	partial
sonar	2	60	206	61	6	11	partial
spambase	2	57	385	91	10	2	partial
spect	2	22	22	53	9	53	full
spectf	2	44	66	10	7	10	full
texture	11	40	335	89	10	0	none
threeOf9	2	9	9	44	8	44	full
twonorm	2	20	200	80	9	1	partial
vehicle	4	18	110	59	10	5	partial
vowel	11	13	108	95	10	3	partial
waveform-21	3	21	203	69	9	1	partial
waveform-40	3	40	337	60	10	1	partial
wdbc	2	30	137	40	9	6	partial
wine-quality-red	6	11	92	93	10	1	partial
wine-quality-white	7	11	95	53	10	1	partial
wine-recog	3	13	47	24	4	24	full
wine-recognition	3	13	47	24	4	24	full
wdbc	2	33	128	35	8	10	partial
xd6	2	9	9	44	9	44	full
zernike	10	47	346	39	9	1	partial

Table 6: Characteristics and scaling result for the remaining 27 models.

B Robustness results for all 54 models

Tables 7 and 8 report the final robustness campaign. Samples is the number of held-out examples analyzed and certified is the subset with a guaranteed winner. Rival median is the median distance to the nearest certified rival. Escape median is the median distance to the complement of the sample’s own certified region. Medians are computed only over certified samples. A timeout means that robustness did not finish within 600 seconds. Not compiled means that no tree prefix was available for the robustness computation.

Dataset	Samples	Certified	Rival median	Escape median	Status
ann-thyroid	50	50	1	1	ok
appendicitis	22	22	4	4	ok
banknote	50	50	1	1	ok
biodegradation	50	36	2	1	ok
breast-cancer	50	50	2.5	2.5	ok
breast-cancer-wisconsin	50	48	3	2	ok
bupa	50	50	1	1	ok
cloud	22	17	1	1	ok
diabetes	50	50	1	1	ok
ecoli	50	48	1	1	ok
factors	—	—	—	—	timeout
fourier	50	38	1	1	ok
glass	41	37	1	1	ok
glass2	33	31	1	1	ok
haberman	50	50	1	1	ok
heart-c	50	50	2	2	ok
heart-h	50	50	3	3	ok
heart-sa	50	50	1.5	1.5	ok
ionosphere	50	50	2	2	ok
iris	30	29	1	1	ok
karhunen	—	—	—	—	timeout
letter	—	—	—	—	not compiled
lupus	18	18	2	2	ok
magic	50	50	1	1	ok
mammo	50	50	1	1	ok
messidor	50	50	1	1	ok
mofn-3-7-10	50	50	1.5	1.5	ok

Table 7: Robustness results for the first 27 models.

Dataset	Samples	Certified	Rival median	Escape median	Status
new-thyroid	43	43	1	1	ok
pendigits	50	50	1	1	ok
phoneme	50	50	1	1	ok
pima	50	50	1	1	ok
ring	50	47	2	1	ok
segmentation	42	36	2	1	ok
seismic_bumps	50	50	6	6	ok
shuttle	50	50	1	1	ok
sonar	42	42	2	2	ok
spambase	50	43	2	1	ok
spect	50	50	3	3	ok
spectf	50	48	4	3	ok
texture	—	—	—	—	not compiled
threeOf9	50	50	1	1	ok
twonorm	50	50	1	1	ok
vehicle	50	47	1	1	ok
vowel	50	41	1	1	ok
waveform-21	50	50	1	1	ok
waveform-40	50	50	1	1	ok
wdbc	50	48	3	2	ok
wine-quality-red	50	50	1	1	ok
wine-quality-white	50	50	1	1	ok
wine-recog	36	36	2	2	ok
wine-recognition	36	36	2	2	ok
wdbc	39	38	3	2	ok
xd6	50	50	1	1	ok
zernike	50	50	1	1	ok

Table 8: Robustness results for the remaining 27 models.